

1. INTRODUCTION

In today's rapidly evolving digital landscape, written communication plays a pivotal role in shaping public opinion, influencing social discourse, and driving decisions. However, the neutrality and objectivity of language are often compromised by biases, whether intentional or unconscious. These biases, coupled with toxic language, can distort perceptions, perpetuate harmful stereotypes, and contribute to the spread of misinformation. Recognizing this challenge, Neutral Note emerges as an innovative tool designed to promote fairness and balance in digital content creation, addressing the urgent need for neutrality in language.

Neutral Note is a cutting-edge language processing solution that analyzes text in real-time to detect both biased and toxic language. By leveraging advanced Natural Language Processing (NLP) techniques and the Detoxify machine learning model, it identifies explicit forms of bias, such as discriminatory language, as well as subtler implicit biases embedded in word choices or phrasing. The tool also flags toxic elements like hate speech or overly aggressive tones, ensuring that users create content that is respectful and constructive. What sets Neutral Note apart is its focus on user education—it doesn't just highlight problematic language but also suggests neutral alternatives, helping users refine their writing while fostering a deeper awareness of their language choices.

Designed with accessibility in mind, Neutral Note is capable of being integrated seamlessly into various text-editing platforms, ensuring that it can be used across diverse workflows. Writers can use it within word processors to improve drafts, journalists can employ it to minimize bias in reporting, and social media managers can craft posts that align with inclusive communication standards. Additionally, organizations can incorporate the tool into corporate communication channels to maintain diversity and inclusion goals. This seamless integration ensures minimal disruption to users' workflows while providing invaluable insights into their text.

The applications of Neutral Note are vast and impactful, extending across multiple industries. In journalism, it ensures that articles are factual and unbiased, preserving the integrity of reporting. In education, it acts as a learning tool for students and educators, encouraging critical thinking about language. For corporate communication, it helps businesses foster positive workplace culture and maintain brand reputation. Content creators in marketing, blogging, and social media benefit from a tool that helps them resonate with diverse and inclusive audiences. Additionally, policymakers can rely on Neutral Note to draft fair and impartial documents, fostering trust in their communications.

Neutral Note's role extends beyond industry-specific applications, as it also serves as a catalyst for societal change. In today's polarized world, where communication is increasingly shaped by social media and instant digital platforms, neutrality is more important than ever. The tool provides an essential mechanism for curbing the spread of harmful rhetoric while nurturing a more thoughtful and respectful public discourse. By offering users a real-time feedback loop on their language, Neutral Note empowers individuals to communicate responsibly, influencing not only their personal interactions but also shaping broader cultural shifts toward inclusivity and empathy.

The technological foundation of Neutral Note is robust, relying on the Detoxify model for detecting toxic language with exceptional accuracy. Advanced NLP techniques enable the tool to perform sentiment analysis, contextual evaluation, and lexical pattern recognition, ensuring comprehensive text analysis. Moreover, Neutral Note's machine learning models continuously evolve, incorporating user feedback and adapting to emerging trends in language usage.

Neutral Note's mission extends beyond detecting and correcting biased or toxic language; it envisions a transformative impact on how we approach communication. By promoting awareness and proactive refinement of language, the tool fosters a culture where words become a medium for unity rather than division. Its emphasis on education encourages users to internalize the principles of fairness and inclusivity, creating a ripple effect that extends beyond individual interactions to society as a whole.

Looking ahead, Neutral Note lays the groundwork for future innovations in language processing. Expanding its bias detection capabilities, supporting multiple languages, and incorporating emotional impact analysis are just a few of its potential advancements. By addressing the pressing need for neutrality in language, Neutral Note not only enhances the quality of communication but also contributes to a more equitable, inclusive, and connected digital society. As technology continues to evolve, Neutral Note remains committed to pushing the boundaries of what is possible in the field of language processing, ensuring that it stays at the forefront of responsible communication tools.

Neutral Note's commitment to fostering responsible communication is not limited to its current capabilities. Future iterations aim to broaden the tool's reach by introducing multilingual support, allowing users worldwide to benefit from its insights. Additionally, advanced emotional tone analysis could enable users to craft messages that not only eliminate bias but also resonate more positively with their audiences, enhancing both personal and professional interactions. This emotional tone feature could also be instrumental in conflict resolution, allowing individuals and organizations to frame messages in ways that foster understanding and cooperation, even in the most sensitive discussions.

By addressing the challenges of biased and toxic language, Neutral Note serves as a catalyst for more inclusive and equitable digital communication. As the world continues to grapple with issues of social justice, equality, and inclusivity, Neutral Note stands as a beacon for change, offering a tool that can help individuals and organizations communicate in ways that are not only more neutral but also more empathetic, respectful, and impactful.

In addition to its practical applications, Neutral Note is committed to evolving with the times. As it gathers more data and feedback from diverse user groups, the tool will refine its algorithms and expand its capacity to recognize and address a wider range of linguistic nuances. This continuous learning process ensures that Neutral Note will always be equipped to handle the complexities of modern language, making it an indispensable ally for those seeking to navigate the challenges of digital communication.

Neutral Note's vision extends into fostering global communication harmony by bridging cultural and linguistic gaps. With planned advancements in multilingual capabilities, the tool aspires to adapt its bias and toxicity detection to account for cultural context and regional sensitivities, ensuring relevance and fairness across diverse audiences. By tailoring its analysis to cultural norms and idiomatic expressions, Neutral Note could become an indispensable resource for fostering cross-cultural understanding in diplomacy, global business, and international media. This expansion not only reinforces its role as a powerful tool for responsible communication but also positions it as a unifying force in an increasingly interconnected world, where the nuances of language can either divide or bring people together.

Ultimately, Neutral Note is not just a tool for detecting biased or toxic language—it is a movement toward a more conscious and considerate way of communicating. In a world where words hold immense power, Neutral Note is leading the charge in ensuring that this power is used responsibly and constructively. By promoting fairness, respect, and inclusivity, Neutral Note is laying the groundwork for a more harmonious and equitable future, one where communication becomes a bridge that connects, rather than a barrier that divides.

2. LITERATURE SURVEY

2.1 Toxicity in Digital Communication

In digital spaces, neutral language is crucial for fostering fair and respectful discourse, especially given its power to shape public opinion and influence personal perceptions. Neutrality helps prevent the reinforcement of societal stereotypes, creating an environment that is inclusive. Language can perpetuate social biases, so tools that promote neutral phrasing can mitigate potential harm by ensuring that communication remains objective and respectful.

Toxic language and bias affect mental health, social inclusivity, and public discourse. Studies show that exposure to biased or toxic language can increase polarization, harm individuals' mental well-being, and contribute to a toxic online environment. This highlights the need for tools that can identify and counteract such language in real-time

2.2 Overview of Toxicity Detection

Early bias and toxicity detection tools used rule-based approaches that were often limited in scope and adaptability. Modern tools leverage machine learning, such as deep neural networks and transformers, allowing for greater contextual understanding and nuance. Pre-trained models like BERT and Detoxify represent advancements in handling complex language structures and sentiments, enabling more accurate bias and toxicity detection.

Key NLP tasks for language neutrality include sentiment analysis, which identifies the emotional tone of language, and bias detection, which identifies subjective or skewed content. These processes are foundational for developing tools that support objective language in digital communication.

2.3 Machine Learning Approaches

Pre-trained models have become instrumental in detecting bias and toxicity due to their ability to process large language corpora and capture nuanced patterns. For instance, Detoxify, which is used in **Neutral Note**, is designed to detect toxic language by learning from diverse datasets. Such models are highly effective for real-time detection but require continual refinement to handle subtle biases.

Models like BERT and GPT are widely recognized for their adaptability and accuracy across contexts. However, they can differ in sensitivity and specificity, with some models better suited to nuanced detection than others. Detoxify, for example, is specifically tuned for toxicity, whereas BERT has broader applications but may require fine-tuning for bias detection.

Detecting bias is challenging due to cultural differences and the subjectivity of language. Many current models struggle with implicit biases that are context-dependent or culturally nuanced, which presents a need for advanced techniques that can handle these subtleties.

2.4 Real-time Detection and Feedback Mechanism

Real-time language processing tools flag biases immediately, allowing users to adjust content before publishing. By using visual indicators such as highlights, underlines, or color-coded suggestions, these tools provide intuitive prompts that guide users toward neutral language choices. This immediate feedback not only enhances the likelihood of creating objective and inclusive communication but also streamlines the writing process by addressing issues as they arise, rather than requiring extensive revisions afterward.

Studies indicate that real-time feedback is significantly more effective than post-creation feedback, as it engages users during the act of writing when they are most receptive to suggestions. This interactive approach helps users internalize best practices, increasing awareness of their linguistic choices and fostering a deeper understanding of how language can influence perception. Over time, this method promotes sustained changes in language use, empowering individuals to write with greater sensitivity and precision.

Moreover, real-time feedback fosters an iterative writing process, where users can experiment with different wordings and immediately see the impact of their adjustments. This dynamic interaction not only improves the quality of the final output but also serves as an educational tool, equipping users with the skills to independently craft neutral and effective communication in the future. By embedding these tools into everyday workflows, writers, educators, and professionals can integrate responsible language practices into their routines, driving lasting change in how we approach written communication.

2.5 Applications in various sectors

Bias detection tools are essential for journalists and media professionals striving to maintain objectivity and credibility. In news reporting, where neutrality is paramount, these tools help identify language that could appear prejudiced or inflammatory, ensuring that narratives remain focused on facts rather than opinion. By fostering balanced and unbiased communication, bias detection strengthens public trust and upholds the integrity of journalism.

Educational institutions and corporations also benefit significantly from bias detection tools. In academic settings, these tools promote inclusive and respectful dialogue, encouraging students and educators to engage with diverse perspectives. For businesses, bias detection supports diversity and inclusion goals by ensuring that internal and external communications are free from unintended prejudice, enhancing workplace harmony and strengthening organizational values.

On social media platforms, bias detection tools play a critical role in moderating content and curbing toxic interactions. By identifying harmful language in real time, these tools foster safer and more constructive online environments, reducing the spread of hate speech and misinformation. As a result, they help create healthier digital communities where users can engage in meaningful, respectful exchanges.

Policymakers and governmental organizations also stand to gain immensely from bias detection tools, as these ensure that official communications, legislative drafts, and public statements are neutral, inclusive, and representative of diverse populations. By minimizing the risk of inadvertently biased language, these tools enhance the credibility and transparency of government communications, fostering trust among constituents. In an era where words can influence public sentiment and policy outcomes, the use of bias detection tools can promote fairness and equity, ultimately strengthening democratic processes and societal cohesion.

3. FEASIBILITY STUDY

3.1 Technical Feasibility

3.1.1 Data Availability

Neutral Note's functionality heavily relies on the availability of comprehensive, high-quality, and annotated language datasets to train and fine-tune the machine learning model for bias and toxicity detection. The use of pre-trained models like Detoxify significantly reduces the need for extensive initial data collection, as these models have been trained on diverse datasets sourced from internet text, social media, and moderated forums. These datasets include nuanced examples of biased and toxic language, enabling the model to capture subtle forms of toxicity, such as microaggressions or implicit bias, alongside explicit forms. To further enhance performance, **Neutral Note** can incorporate anonymized user inputs during its operation, refining the model based on real-world usage patterns. This iterative learning process ensures that the model remains adaptive and relevant to evolving language trends, including emerging slang or cultural contexts. Additionally, access to domain-specific datasets, such as those related to journalism or corporate communication, could enable more tailored functionality for specific user groups.

3.1.2 Computational Resources

Building and maintaining a high-performance deep learning model for bias and toxicity detection requires robust computational infrastructure. Modern cloud computing platforms, such as Google Cloud, Amazon Web Services (AWS), and Microsoft Azure, offer scalable solutions that eliminate the need for costly on-premise servers. These platforms provide access to **Graphics Processing Units (GPUs)** and **Tensor Processing Units (TPUs)**, which accelerate model training and inference tasks, ensuring the system can process large volumes of data in real time. Cloud services also support elastic scaling, allowing Neutral Note to handle fluctuations in demand without compromising performance. For instance, during peak usage periods, additional resources can be allocated dynamically to ensure responsiveness. Moreover, cloud providers offer integrated tools for monitoring, optimization, and cost management, enabling Neutral Note to balance performance and affordability effectively.

3.1.3 Model Development

Neutral Note's core relies on its ability to analyze text for bias and toxicity in real-time, leveraging Detoxify, a pre-trained model based on powerful architectures like **BERT (Bidirectional Encoder Representations from Transformers)** and **GPT (Generative Pre-trained Transformer)**. These models excel at contextual understanding, making them adept at detecting nuanced and implicit forms of bias. By utilizing transfer learning, Detoxify can be further fine-tuned to cater to Neutral Note's specific requirements, such as identifying biases in formal or domain-specific content like news articles or corporate emails. This approach significantly reduces development time and costs while ensuring high accuracy. Additionally, the iterative fine-tuning process incorporates real-world feedback, enabling the model to adapt continuously to new contexts, emerging biases, and linguistic trends. Neutral Note also prioritizes interpretability, ensuring users receive not just results but meaningful explanations behind flagged biases, fostering transparency and trust.

3.2 Operational Feasibility

3.2.1 Data Preprocessing

Preprocessing is an essential step in preparing the text data for analysis. The input text, whether from social media posts, articles, or emails, must undergo several preprocessing steps such as tokenization, normalization, and noise reduction to ensure that it is in a form suitable for model evaluation. Automation of these tasks ensures that users can receive real-time feedback with minimal delay. By integrating efficient data pipelines, **Neutral Note** can maintain high performance even as the volume of text data increases. The automation of data preprocessing also reduces manual intervention, improving the scalability and operational efficiency of the system.

3.2.2 Model Training and Evaluation

Effective model training requires careful selection of the model architecture, hyperparameter tuning, and robust evaluation to ensure the tool provides accurate and reliable feedback. The model training process will be iterative, using performance metrics such as accuracy, precision, and recall to assess the model's effectiveness in identifying biases and toxicity. Given that the model is pre-trained, much of the heavy lifting has already been completed, allowing us to focus on evaluating how the model performs on specific datasets relevant to **Neutral Note's** target users.

3.3 Economic Feasibility

3.3.1 Development Costs

Neutral Note's reliance on pre-trained models like Detoxify reduces initial development costs significantly, as much of the foundational work in training the model has already been completed. However, expenses will still be incurred for activities like fine-tuning the model, integrating it into user-friendly interfaces, and maintaining cloud-based infrastructure. Open-source libraries such as **Hugging Face Transformers** and **PyTorch** can be leveraged to minimize software licensing fees. Cloud platforms offer flexible pricing models, such as **pay-as-you-go** or reserved instances, enabling the project to manage computational costs effectively during both development and deployment phases.

3.3.2 Implementation Costs

The successful implementation of Neutral Note requires a strategic approach to raise awareness and onboard users. Marketing efforts, including targeted campaigns for writers, journalists, and corporations, will incur costs related to content creation, advertisements, and promotional activities. Additionally, user onboarding processes, such as providing tutorials, customer support, and technical documentation, will require investments in human resources and training materials. While these costs are necessary to ensure smooth adoption, they can be minimized through scalable solutions such as automated onboarding tools, FAQs, and video tutorials. Partnerships with media organizations, educational institutions, and corporate networks can further reduce implementation expenses by facilitating bulk adoption.

3.3.3 Benefits

The tool's ability to offer real-time feedback will result in reduced editing time, enabling writers to produce more neutral content faster. Additionally, it can help mitigate potential legal and reputational risks associated with biased or discriminatory language in public communications.

4. SYSTEM REQUIREMENTS

4.1 Existing System

The field of language bias and toxicity detection in NLP has seen many approaches, primarily driven by rule-based systems or simple machine learning models that have difficulty scaling to modern digital communication needs. Existing systems such as Instagram, Facebook often lack real-time processing capabilities and the advanced, nuanced detection of subtle biases. While some tools offer sentiment analysis or basic toxicity detection, none provide interactive feedback, real-time rephrasing, or educational insights to help users understand language neutrality.

4.2 Proposed System

Neutral Note is a cutting-edge solution designed to overcome the limitations of traditional bias detection systems by providing real-time feedback on biased or toxic language in digital communication. Unlike conventional tools that focus solely on identifying overt toxicity, Neutral Note delves deeper into the nuances of language, detecting both explicit and implicit biases that can unconsciously shape perceptions. Leveraging advanced Natural Language Processing (NLP) techniques, it employs Detoxify—a state-of-the-art pre-trained model fine-tuned on diverse datasets—to accurately assess text for subtle forms of bias, toxicity, and stereotypes.

One of the standout features of Neutral Note is its ability to provide immediate and actionable visual feedback. Users are alerted to problematic terms or phrases through intuitive highlights or annotations, making it easy to pinpoint areas of concern. Alongside these alerts, the tool offers carefully curated suggestions for neutral rephrasing, empowering users to modify their language while maintaining the intended message. This ensures that content remains both inclusive and effective, promoting a respectful tone without sacrificing clarity or impact.

Optimized for real-time interaction, Neutral Note is designed to integrate seamlessly into various digital environments, such as text editors, content management systems, and social media platforms. This adaptability allows writers, journalists, educators, and professionals to enhance their content directly within their workflows. By fostering greater awareness of language choices and providing tools to refine communication, Neutral Note supports its users in creating objective, respectful, and impactful content tailored for diverse audiences. Through its innovative approach, Neutral Note not only addresses the challenges of bias detection but also sets a new standard for promoting inclusivity and objectivity in digital communication.

4.3 System Requirements

The development and deployment of **Neutral Note** require specific hardware and software configurations to ensure optimal performance and compatibility

4.3.1 Software Requirements

Programming Languages: Python is the primary language for the core NLP functionalities, leveraging libraries such as Transformers and PyTorch for model implementation and Hugging Face for Detoxify.

Frameworks and Libraries:

Django: For backend development, managing the application structure, user sessions, and database interactions. Version is 4.1.13

User Interface: For frontend development, we have used HTML, CSS, JavaScript, jQuery.

Detoxify: A pre-trained model used to detect toxicity and bias, incorporated into the system for nuanced language analysis. There are 3 variants i.e., original, unbiased, multilingual of Detoxify Model. Here we are using ‘unbiased’ version which is capable of understanding the context more in a text.

Database: MariaDB (XAMPP) to store user data, logs of text entries, flagged content, and rephrased suggestions. This allows tracking usage patterns and model improvements. Libraries such as PyMySQL or mysqlclient (2.2.4) are used so as to connect the application to MariaDB.

OpenAI’s API: This is called so as to rephrase the text in case it is classified as any of the categories of Detoxify Model.

Encoding and Decoding: PyJWT is used so as to encode the password for user’s security while storing in the database.

4.3.2 Hardware Requirements

Processor: Minimum dual-core processor, although a quad-core processor is recommended for faster model processing.

RAM: At least 8GB of RAM is required to run the Detoxify model effectively. For smoother performance, especially for real-time processing of large datasets, 16GB RAM is recommended.

Storage: 50GB of storage is sufficient for the codebase, model files, user data, and logs. SSD storage is recommended for faster data access and processing.

4.4 Requirements Definition

Neutral Note is designed to ensure reliability and accessibility for end-users by defining a set of functional and non-functional requirements.

4.4.1 Functional Requirements

Text Analysis and Detection: Analyze user input in real-time to detect biased or toxic language, flagging explicit or implicit biases.

Visual Feedback Mechanism: Provides neutral rephrasing suggestions.

User Interface: An interactive, intuitive interface for users to view flagged content, access suggestions, and revise text seamlessly.

4.4.2 Non-Functional Requirements

Performance: Real-time processing of text inputs with a minimal delay for immediate feedback.

Scalability: Capability to handle increasing numbers of users and text inputs without degradation in performance.

Usability: A straightforward user interface that is accessible to non-technical users, ensuring ease of use.

Security: User data protection with secure storage and handling, ensuring compliance with data privacy regulations.

Maintainability: Modular code design to facilitate future updates, feature additions, and bug fixes.

5. SYSTEM DESIGN

5.1 Architecture

The **Neutral Note** project architecture incorporates a combination of user interaction layers, backend processing, and machine learning (ML) integrations for real-time analysis and rephrasing. Here's an overview of its architecture based on your workflow:

- **Frontend Layer (User Interface):** The entry point for users to interact with Neutral Note. It comprises:
 - **Login Interface:** The initial module, which allows users to sign up or sign in, ensuring authenticated access.
 - **Toxicity Analysis Interface:** Once logged in, users are presented with an input field where they can submit text for analysis. This interface displays results and rephrasing suggestions.
- **Backend Processing:** The core processing logic resides in the backend, which performs the following:
 - **Text Analysis:** Using the Detoxify ML model, the backend detects toxic language and categorizes it based on various types of toxicity.
 - **Rephrasing:** For flagged content, the backend sends the toxic segments to the OpenAI's API, which returns rephrased, neutralized suggestions.
- **Machine Learning Models and APIs:**
 - **Detoxify Model:** A pre-trained ML model that detects toxic language, identifying both overt and subtle toxicity in user inputs.
 - **OpenAI GPT-4 Turbo:** Responsible for rephrasing toxic text. Detoxify flags issues, and OpenAI's API helps refine flagged language into a more neutral form.
- **Database:** Stores user data, login credentials, and potentially past interactions or rephrased outputs. With XAMPP serving as the database and server environment, Neutral Note leverages its integrated components:
 - **MySQL Database (MariaDB):** Manages and stores user data efficiently in a relational format, allowing secure, structured queries.
 - **Apache Server:** Hosts the application locally, processing user requests and serving dynamic pages in response to toxicity analysis and rephrasing functions.

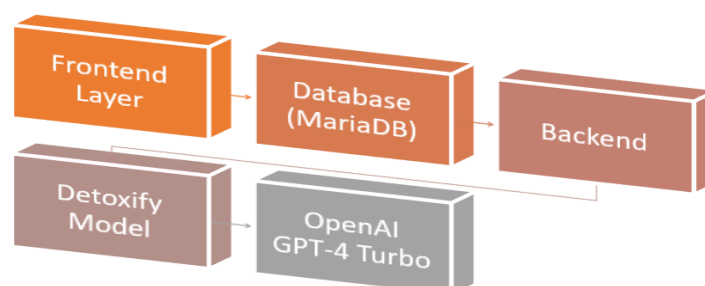


Fig 5.1.1: Architecture

5.2 UML Diagrams

5.2.1 Use Case Diagrams

A use case diagram is a UML diagram that visually represents the interactions between users (called "actors") and a system to show the system's functionality. It outlines the main functions (called "use cases") that the system provides, showing what users can do and how the system responds.

✚ **Actors:** The user interacts with the system for signing up, signing in, and entering text.

✚ **Use Cases:**

- **Sign Up** and **Sign In** represent user authentication.
- **Enter Text for Analysis** lets the user input text.
- **Analyze Toxicity** processes the entered text.
- **Rephrase Toxic Text** is included if the text is detected as toxic.
- **Display Results** show the outcome of the analysis or rephrasing.

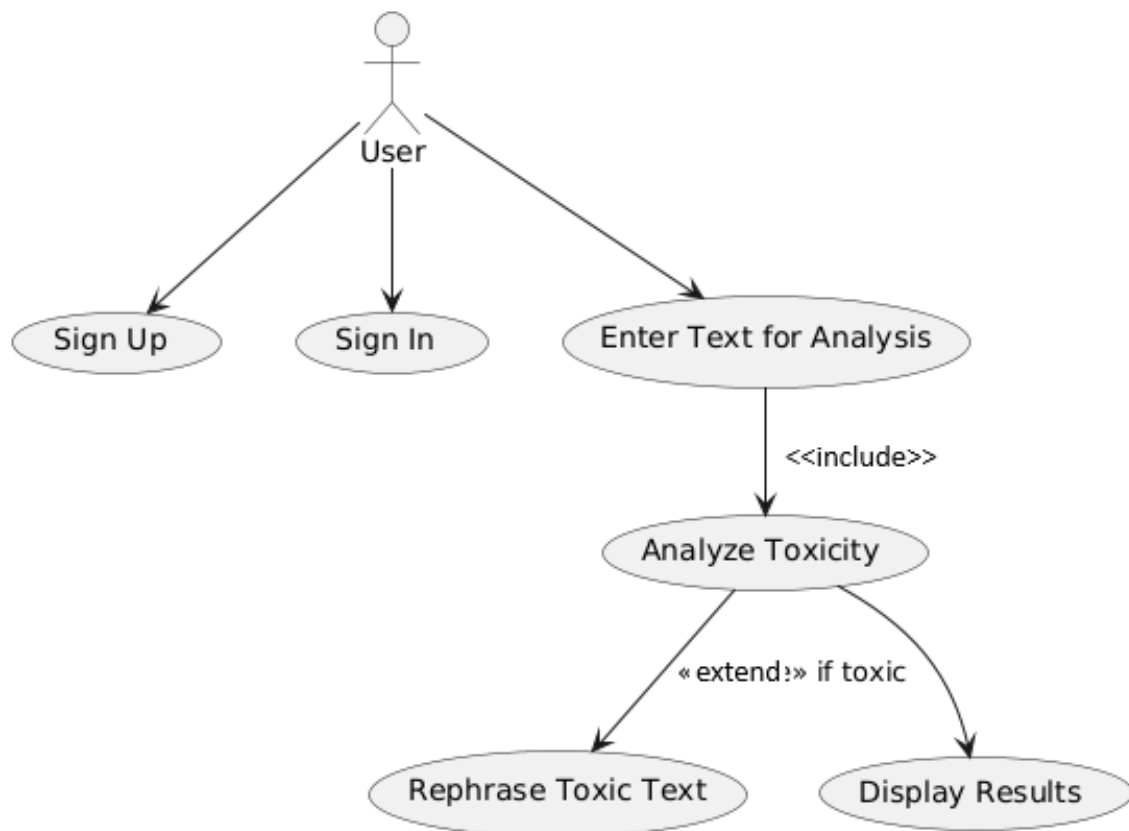


Fig 5.2.:1 Use Case Diagram

5.2.2 Class Diagram

A class diagram is a type of UML diagram that shows the structure of a system by depicting its classes, attributes, methods, and the relationships between classes. It helps visualize the organization and design of a software system in an object-oriented approach.

- + User interacts with **UserInterface** to either log in or use the toxicity detection tool.
- + **LoginManager** handles authentication tasks like sign-up and sign-in, using **Database** to validate or save user details.
- + **ToxicityAnalysis** receives text input from **UserInterface** and analyzes it for toxicity.
- + If toxicity is detected, **ToxicityAnalysis** calls **RephraseService** to provide a neutral rephrasing.
- + **RephraseService** relies on **OpenAI API** to process and return a rephrased version of the text.
- + **UserInterface** displays results, including flagged categories, original text, and rephrased output.

Interactions and Relationships:

- **User** interacts with the **UserInterface**, which serves as the point of access to the system's features.
- **UserInterface** communicates with **LoginManager** for authentication and with **ToxicityAnalysis** for text processing tasks.
- **LoginManager** relies on the **Database** to validate user credentials or store new account details.
- **ToxicityAnalysis** processes input text and, if toxicity is detected, invokes **RephraseService** for rephrased suggestions.
- **RephraseService** depends on **OpenAI API** to generate contextually appropriate and neutral outputs.
- **UserInterface** displays the results, including flagged categories, analysis summaries, and rephrased text, ensuring the user has actionable feedback.

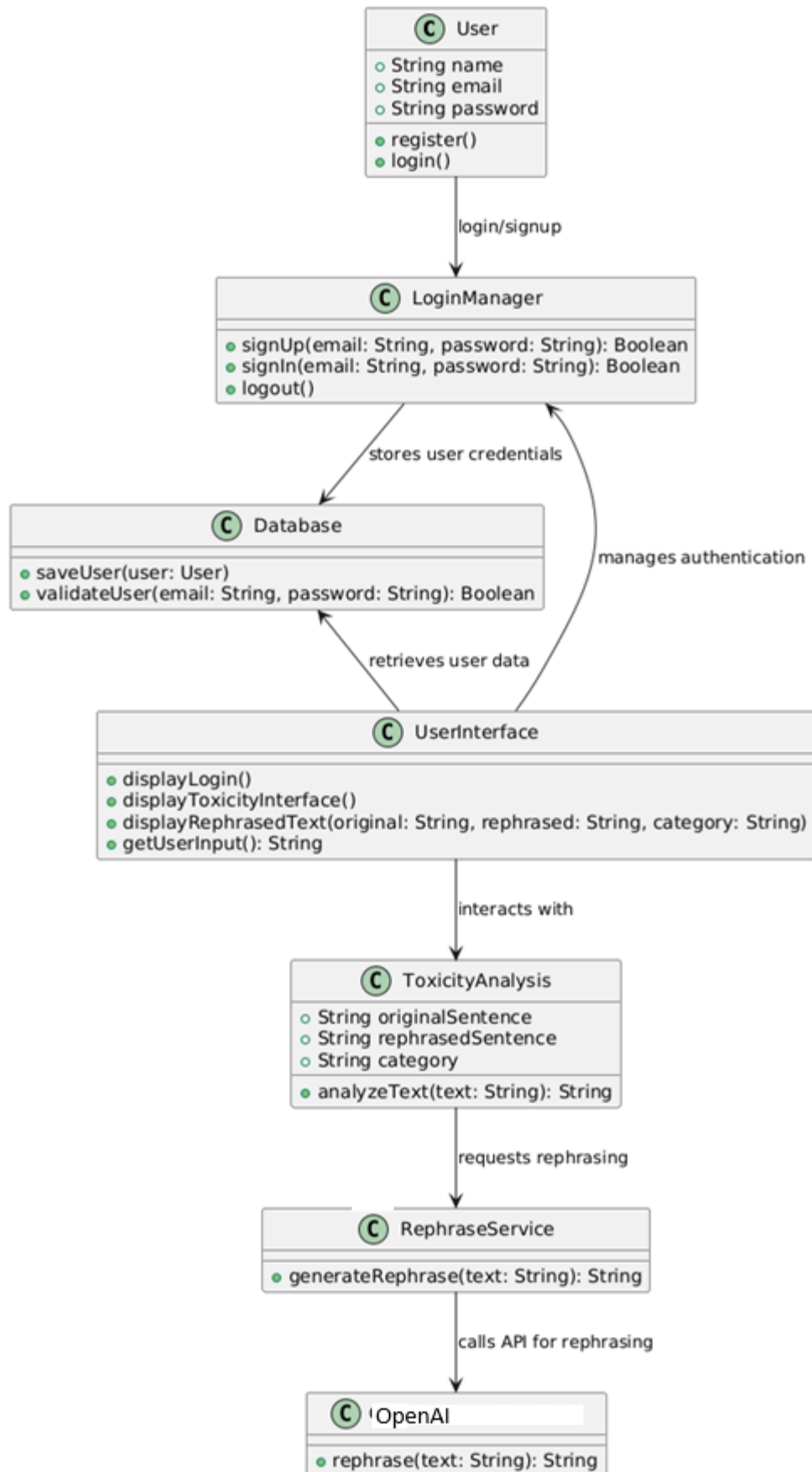


Fig 5.2.2: Class Diagram

5.2.3 Object Diagram

An object diagram consists of several key components:

Objects: Represent instances of classes from the class diagram. Each object has a name and its state is depicted through its attributes.

- ✚ **User Object:** This object could include attributes like `userName`, `email`, and `password`. This object is used for login authentication.
- ✚ **ToxicityService Object:** It may contain methods like `checkTextForToxicity()`, and attributes like `detectedToxicity`, which represent the detected level of toxicity after the user inputs text.
- ✚ **RephraseService Object:** This object, which uses
- ✚ **Generative AI API**, could be involved in the rephrasing process. Its attributes might include `originalText`, `rephrasedText`.

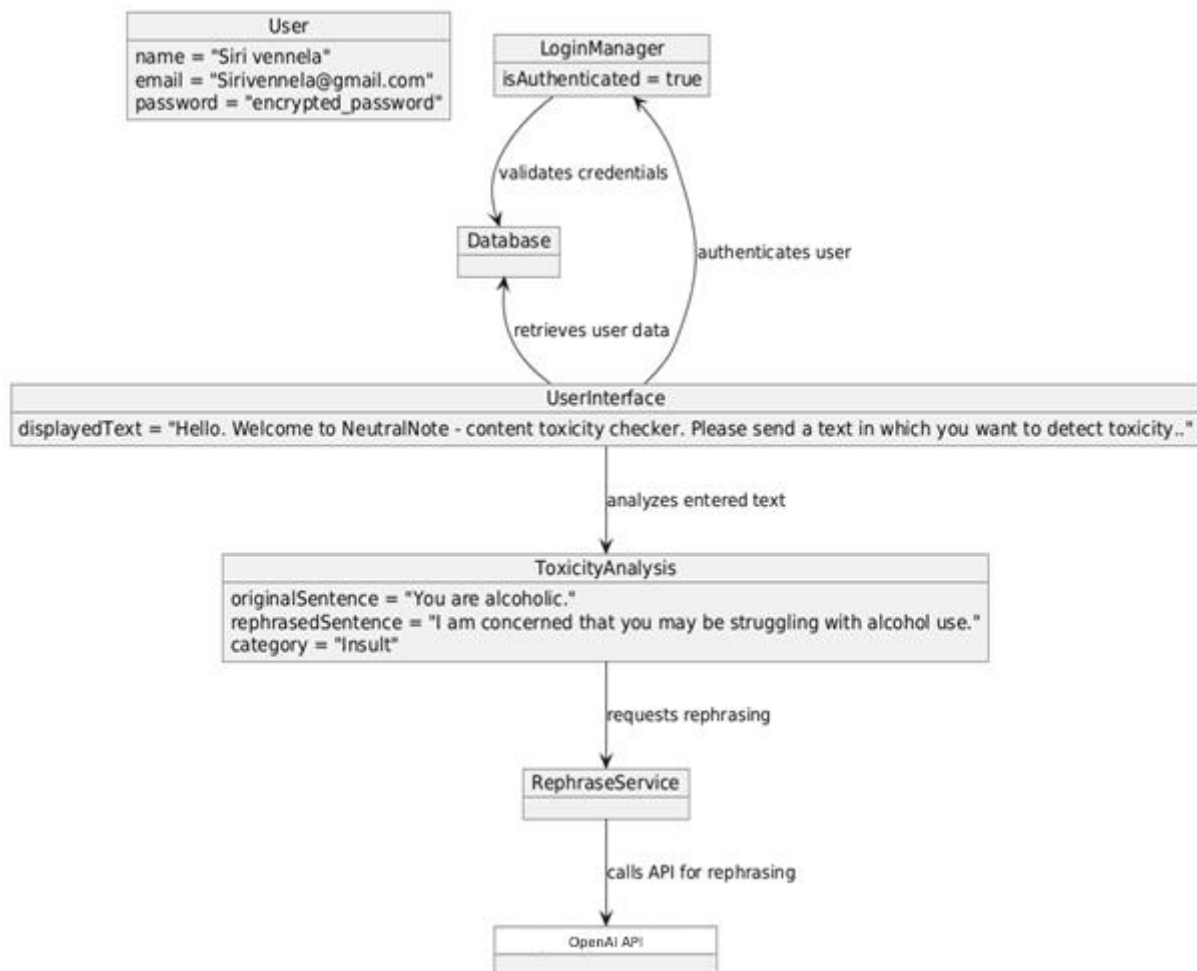


Fig 5.2.3: Object Diagram

5.2.4 Sequence Diagram

A sequence diagram is a type of UML (Unified Modeling Language) diagram that models the dynamic behavior of a system by showing how objects or components interact with each other over time. It represents the sequence of messages exchanged between different entities (such as actors, systems, or objects) in a specific scenario or process.

Sequence: The order of operations from top to bottom follows the sequence in which each action or message occurs.

✚ **Actors and Lifelines:** User, LoginPage, AuthController, UserDB, ToxicityController, DetoxifyModel, and OpenAI API represent the system components and actors involved.

✚ **Messages:** Arrows (->) represent interactions and method calls between objects.

Return Messages: Dashed arrows (-->) show the responses or returns from methods.

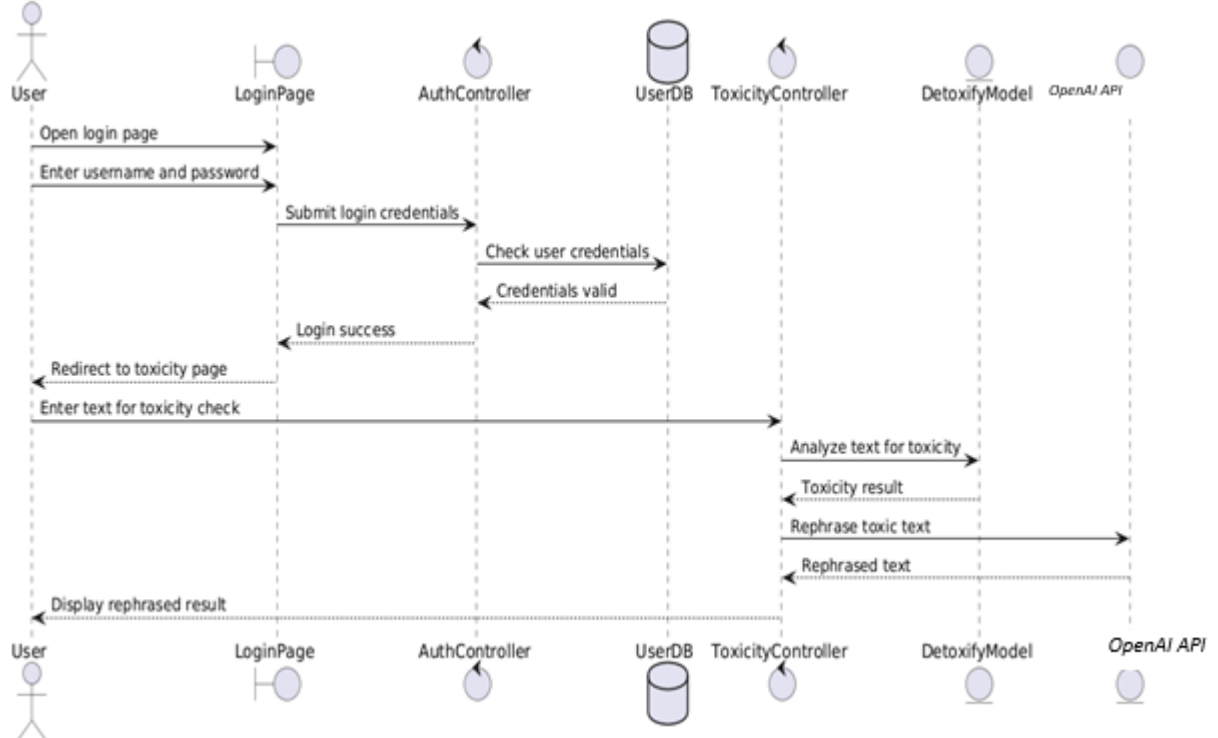


Fig 5.2.4: Sequence Diagram

5.2.5 Activity Diagram

An activity diagram is a UML diagram that visually represents the workflow or sequence of activities in a system or process. It shows the flow from one activity to another, including decision points, parallel actions, and start/end points. Activity diagrams are useful for modeling business processes, system functions, and complex workflows, helping to clarify how tasks progress within a system.

- ✚ **Start:** The process begins when the user opens the website or application interface. They are presented with the option to log in or sign up.
- ✚ **Sign Up/Sign In:** If the user is new, they sign up; if not, they sign in.
 - **New User:** If the user is new, they proceed to the Sign Up activity, providing details such as a username, email, and password. Upon successful registration, they are logged into the system.
 - **Returning User:** If the user already has an account, they proceed to the Sign In activity. Credentials are validated via the LoginManager, and the user is granted access if authentication is successful.
 - **Error Handling:** If login or registration fails (e.g., invalid credentials or duplicate accounts), the user is prompted to correct the input or try again.
- ✚ **Access Toxicity Page:** Once authenticated, the user navigates to the toxicity detection interface, where they can input the text they wish to analyze.
- ✚ **Input Validation:** The system checks the input for basic criteria (e.g., non-empty text, valid characters) before proceeding to analysis. If the input is invalid, an error message is displayed, and the user is prompted to re-enter the text.
- ✚ **Process Text:** The **ToxicityAnalysis** component receives the input and processes it using a machine learning model, such as Detoxify, to detect toxic elements. The analysis checks for various toxicity categories, including hate speech, abusive language, and overly aggressive tones.
- ✚ **Rephrase if Toxic:** The system invokes the **RephraseService**, which interacts with OpenAI's API to generate a neutral, non-toxic version of the input text.
- ✚ **Display Results:** The results are displayed on the interface, showing the original text, flagged toxic categories (if any), and the rephrased version.

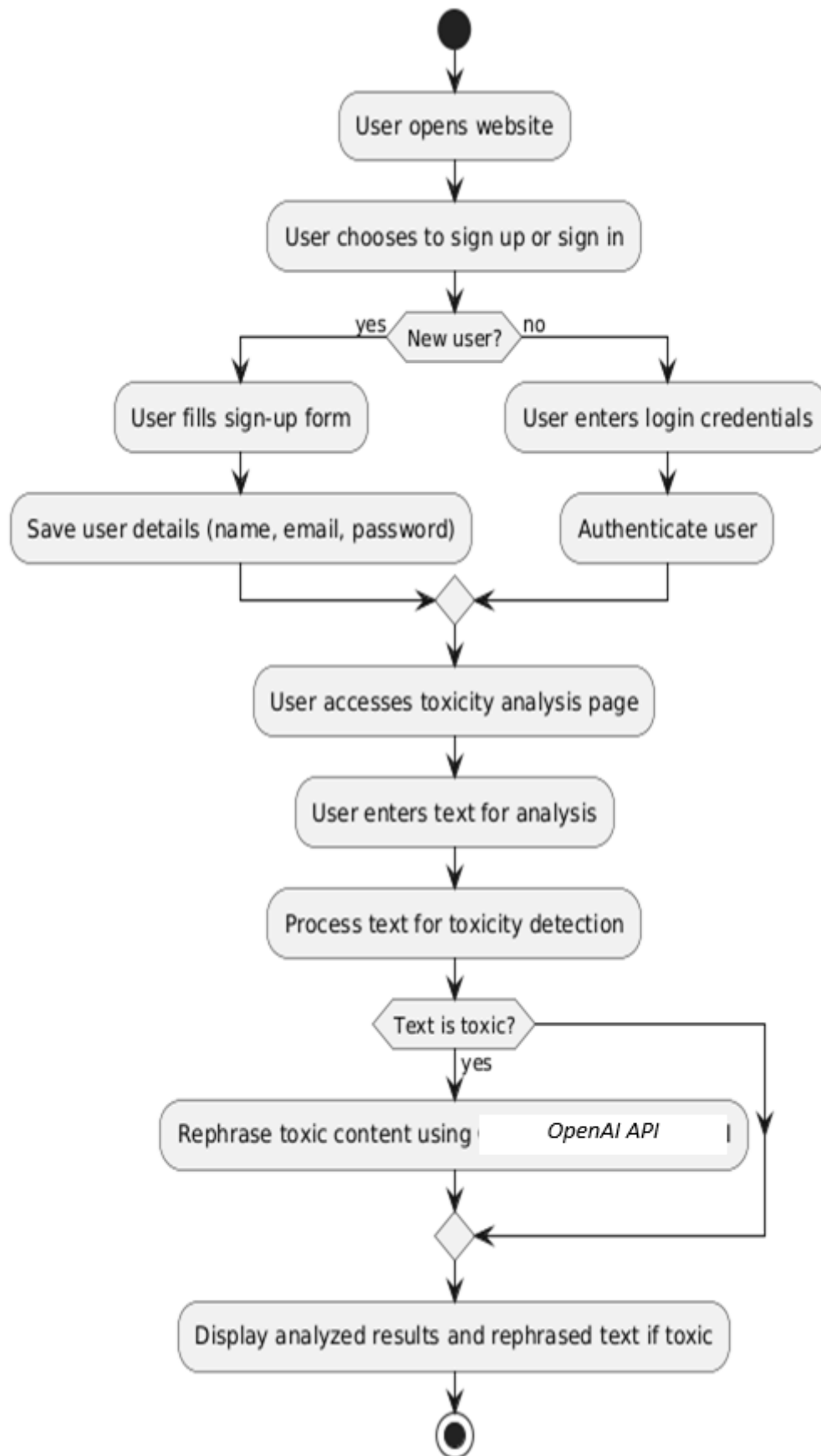


Fig 5.2.5: Activity Diagram

5.2.6 State Diagram

A state diagram is a UML diagram that represents the different states of an object in a system and the transitions between those states in response to events. It provides a clear visualization of how an object behaves over time, showing its journey through various states and the events that trigger state changes. This makes state diagrams especially useful for modeling the dynamic behavior of systems with multiple states and conditions.

- ✚ **Login State:** The **Login State** serves as the starting point. Users can either sign up as new users or sign in if they have an account. Successful sign-up transitions to the **SignIn State**, while failed attempts return to the **SignUp State** for corrections. Similarly, successful sign-in moves to the **Main Interface State**, while failed attempts loop back for retry.
- ✚ **SignUp:** In the **SignUp State**, new user details are collected and validated. Successful registration transitions to **SignIn**, while errors such as weak passwords or duplicate emails keep the user in the current state with prompts to fix issues.
- ✚ **SignIn:** The **SignIn State** authenticates returning users. Providing valid credentials leads to the **Main Interface State**, granting access to core features. Invalid credentials loop the user back to retry or recover their details.
- ✚ **Main Interface:** The **Main Interface State** allows users to input text for toxicity analysis. Upon submission, the state transitions to **Toxicity Analysis State**. Users can also log out from here, returning to the **Login State**.
- ✚ **Toxicity Analysis State:** The entered text is processed for toxicity.
 - If toxic, the text goes to **Rephrasing**.
 - If non-toxic, it goes directly to **DisplayResult**.
- ✚ **DisplayResult State:** The **Display Result State** shows the final analysis to the user. It includes the original text, flagged toxic elements, and rephrased suggestions if applicable. Users can analyze more text or log out from here.

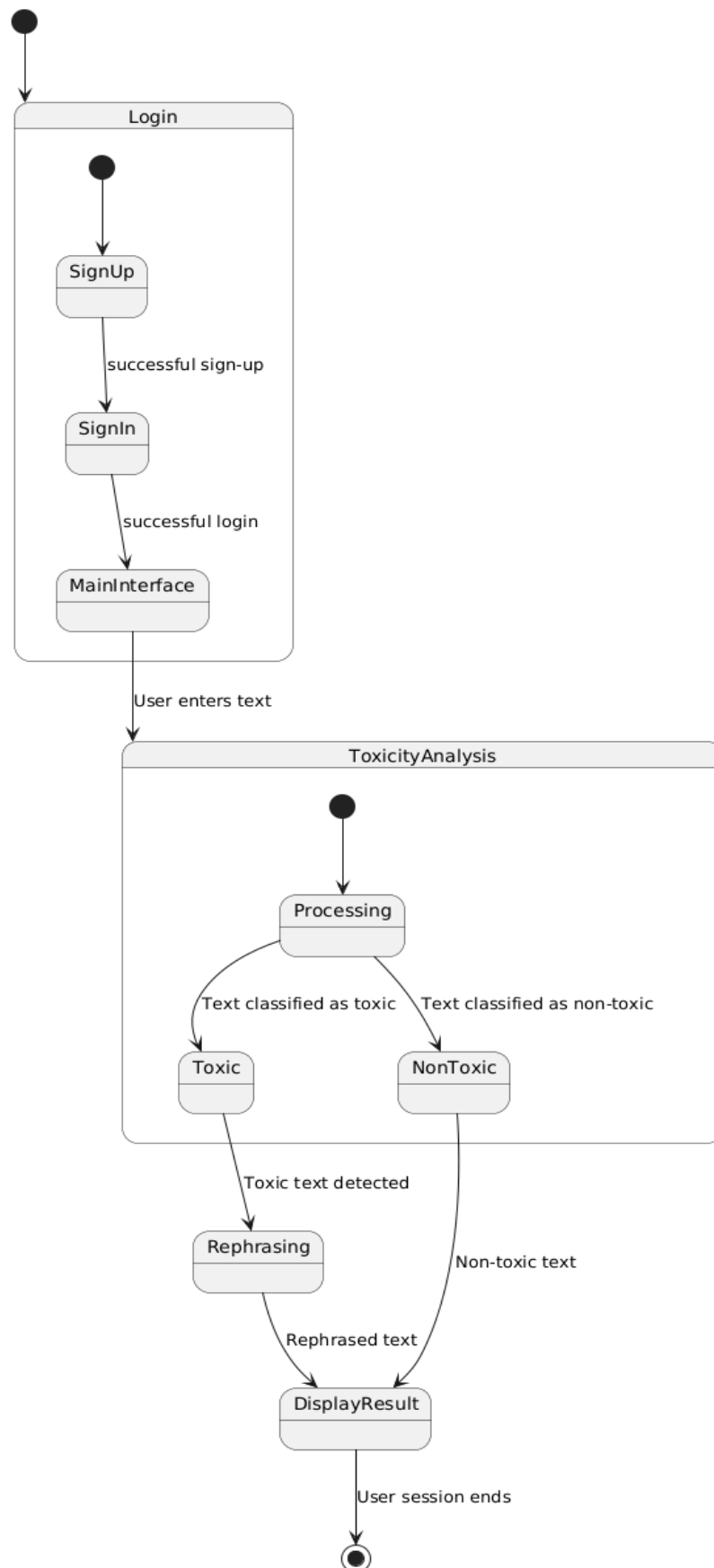


Fig 5.2.6: State Diagram

5.2.7 Component Diagram

A component diagram is a UML diagram that shows the organization and dependencies among various software components in a system. It represents how components, such as libraries, executables, or modules, interact and work together. Each component typically represents a significant piece of functionality or code that is independently deployable. Component diagrams help in understanding the structure of complex systems by illustrating the relationships between different parts of the software.

- ✚ **Web Server:** Contains the main components responsible for user interaction and toxicity analysis, including the Login Service, Toxicity Analysis Service, Rephrase Service, and Display Interface.
- ✚ **Database Server:** Holds the Database component, where user credentials are stored and validated.
- ✚ **Components:**
 - **Login Service:** Handles user login and sign-up requests.
 - **Toxicity Analysis Service:** The core service that processes text submitted by users and detects toxicity.
 - **Rephrase Service:** If the text is toxic, it interacts with the OpenAI's API to rephrase the text.
 - **Display Interface:** The user-facing component that displays results of the toxicity analysis and rephrasing.
- ✚ **Interactions:**
 - The User interacts with the system through the Login Service, submits text to the Toxicity Analysis Service, and views results on the Display Interface.
 - The Toxicity Analysis Service communicates with the Rephrase Service and the Display Interface to handle toxic text detection and result presentation.
 - The Login Service checks user credentials in the Database.

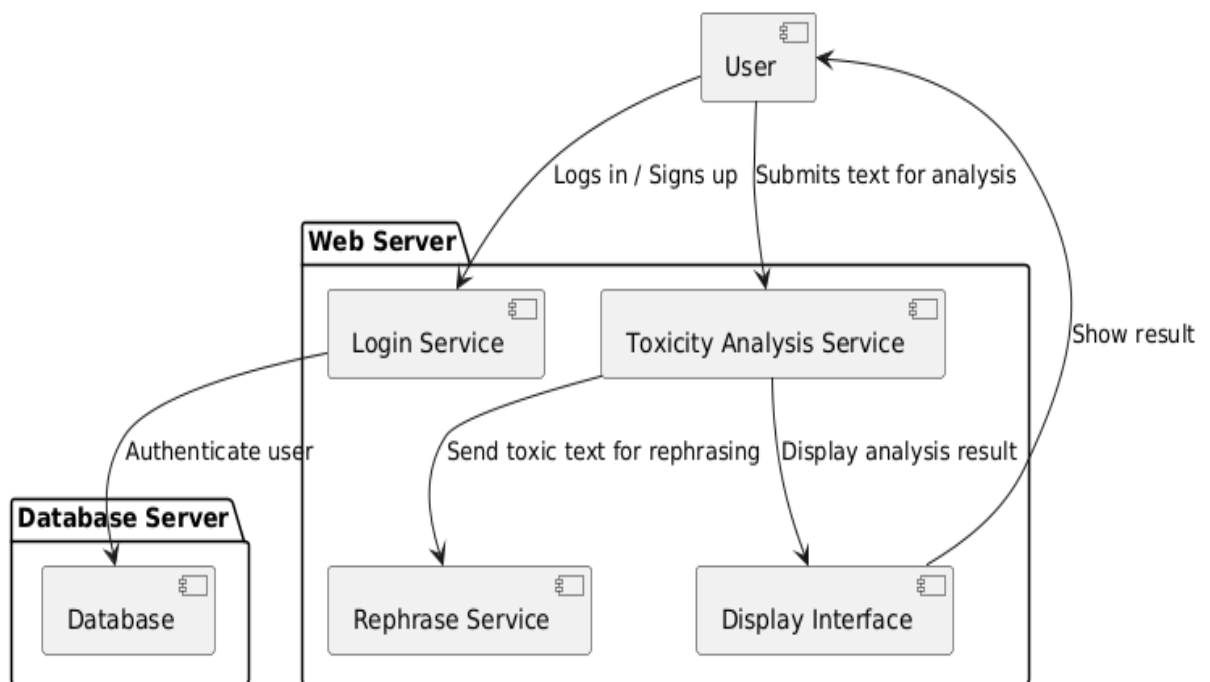


Fig 5.2.7: Component Diagram

5.2.8 Deployment Diagram

A deployment diagram is a UML diagram that illustrates the physical architecture of a system, showing how software components are deployed on hardware nodes. It displays the configuration of runtime elements such as servers, devices, and network connections, helping to visualize the system's infrastructure and its deployment environment.

Actors and Nodes:

- ✚ **User:** The actor interacts with the web server to log in and submit text for toxicity analysis.
- ✚ **Web Server:** Contains several artifacts representing the core services (Login Service, Toxicity Service, Rephrase Service, Display Interface).
- ✚ **Database Server:** The database that stores user credentials.

Communication Flow:

- ✚ **Login and Authentication:** The User logs in or signs up through the Display Interface, and the Login Service validates credentials with the Database Server.
- ✚ **Toxicity Analysis:** Authenticated users submit text, which the Toxicity Service evaluates for explicit and implicit biases using the Detoxify model
- ✚ **Rephrasing and Feedback:** Flagged toxic content is sent to the Rephrase Service, which generates a neutral rephrased version for the User.

Result Display: The Display Interface summarizes the analysis with flagged terms, rephrased text, and a toxicity score.

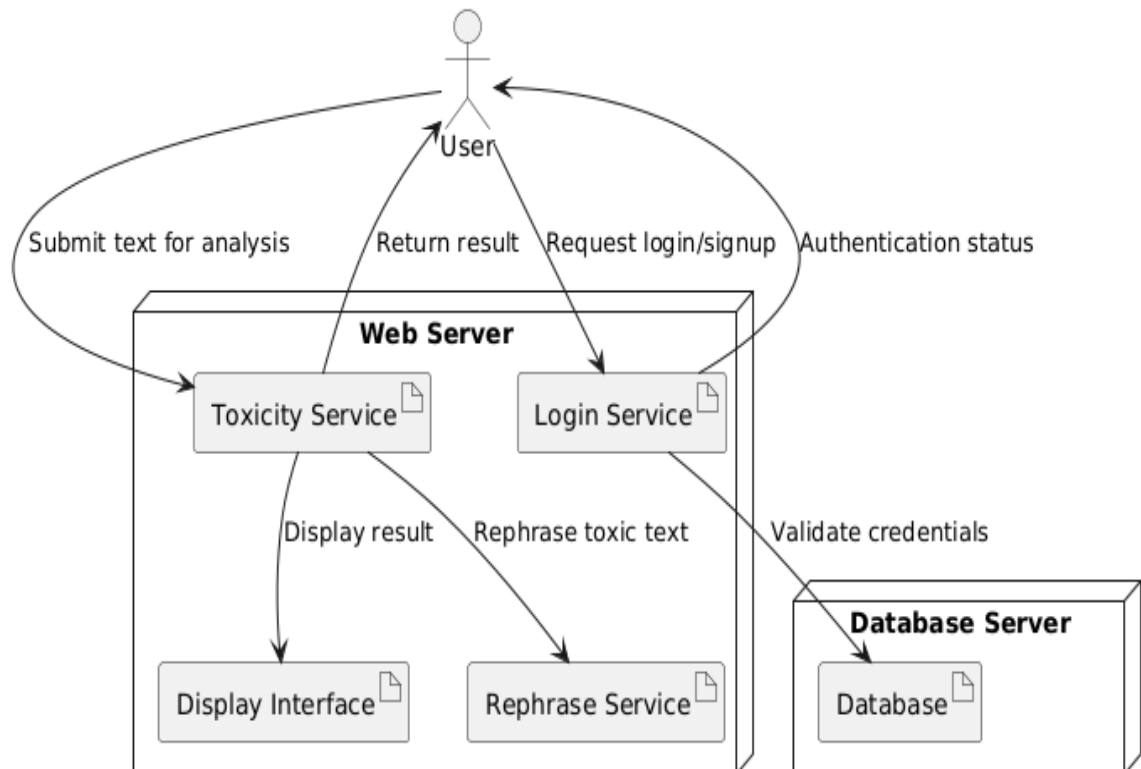


Fig 5.2.8: Deployment Diagram

6. SOFTWARE IMPLEMENTATION

6.1 Technologies

Neutral Note is built on a robust stack that supports efficient text analysis and real-time rephrasing capabilities. Key technologies include:

- **Backend:** Python and Django framework for handling HTTP requests, user authentication, and interfacing with the Machine Learning model for toxicity detection.
- **Machine Learning Models:** The project leverages a pre-trained ML model, Detoxify, to identify toxic language, and OpenAI's GPT-4 Turbo for real-time rephrasing of identified toxic sentences.
- **Frontend:** HTML, CSS, and JavaScript are used for the user interface, along with AJAX and jQuery for asynchronous data requests.
- **Database:** MySQL, managed through XAMPP(MariaDB), stores user details including names, emails, and passwords.
- **API Integration:** OpenAI's GPT-4 Turbo API is integrated to handle real-time rephrasing requests, ensuring smooth interaction and response times for users.

6.2 Modules

The project is divided into the following primary modules:

- **Login Module:**
 - Manages user authentication, including sign-up, login, and session handling.
 - Ensures only authenticated users access the toxicity analysis and rephrasing functionalities.
- **Toxicity Detection Module:**
 - Handles the core functionality of detecting toxic language within user-submitted text.
 - Calls the Detoxify model for toxicity analysis and categorization of toxic language.
- **Rephrasing Module:**
 - Uses OpenAI's GPT-4 Turbo API to rephrase toxic sentences, ensuring they align with neutral communication standards.
 - Integrates the rephrased text back into the main text before displaying the final result to the user.
- **Display and Feedback Module:**
 - Presents analysis results and suggested rephrases in a user-friendly format.
 - Provides real-time feedback to enhance user experience and reinforce the educational aspect of neutral language.

6.3 Methodology

6.3.1 Detoxify Model

The Detoxify model employs a **Transformer-based architecture** (such as BERT, RoBERTa, or DistilBERT), which is widely known for its efficiency in understanding natural language. The key algorithmic components driving the classification process are as follows:

➤ **Transformer Architecture:**

- At the core, Detoxify leverages the transformer model, which uses **self-attention mechanisms**. These mechanisms enable the model to attend to all parts of a text simultaneously, capturing long-range dependencies and context across words. Unlike traditional models like Recurrent Neural Networks (RNNs), transformers process words in parallel, making them faster and more scalable (Vaswani et al., 2017).
- Transformers work by embedding words into high-dimensional vectors that encapsulate their semantic meaning and context. The model then passes these embeddings through multiple layers of self-attention and feed-forward networks.

➤ **Pre-training and Fine-Tuning:**

- **Pre-training:** Detoxify uses a pre-trained transformer model (e.g., BERT) that was initially trained on a large corpus of text, such as Wikipedia, books, and news articles. This process allows the model to learn grammar, word meanings, and general language structure, without any specific task in mind.
- **Fine-Tuning:** After pre-training, Detoxify undergoes **fine-tuning** on a labeled toxicity dataset (size = 906.7 MB). During this phase, the model learns to identify patterns and linguistic features specific to toxic language (e.g., hate speech, offensive content). The model is exposed to texts labeled as toxic or non-toxic, enabling it to adjust its weights and biases to improve classification accuracy for toxicity detection.

➤ **Classification and Output Generation:**

- The model employs **softmax activation** at the final output layer. This step converts the raw output logits (real-valued scores) into a probability distribution across possible categories.
- The model is typically set to output the category with the highest probability. For instance, a text could have a 0.85 probability of being toxic and a 0.15 probability of being non-toxic. If the threshold is set at 0.5, the text would be classified as toxic.

➤ **Loss Function:**

- The model uses **cross-entropy loss** during training to minimize the difference between predicted probabilities and actual labels. Cross-entropy loss is commonly used in classification tasks, especially in multi-class and binary settings, and helps to refine the model's parameters to make better predictions.

➤ Toxicity Categories:

The Detoxify model may assign labels to text in several toxicity categories such as:

- toxicity
- severe_toxicity
- obscene
- threat
- insult
- identity_attack
- sexual_explicit

Identity labels used:

- male
- female
- homosexual_gay_or_lesbian
- christian
- jewish
- muslim
- black
- white
- psychiatric_or_mental_illness

➤ Token Limit and Processing

A critical consideration when using transformer-based models is the **token limit** — the maximum number of tokens (words or subwords) the model can process in a single forward pass. For Detoxify, based on models like BERT, the typical **token limit** is **512 tokens**. This means that any input longer than 512 tokens must be truncated or split into smaller chunks. Each token represents a meaningful part of the input text, and the model uses them to build a contextual representation.

- **Tokenization Process:** Detoxify converts the input text into tokens using a **tokenizer** (often **WordPiece** or **Byte-Pair Encoding**). These tokens are then embedded into high-dimensional vectors, which the transformer model processes to extract contextual information.
- **Limitation:** When the input exceeds 512 tokens, the text needs to be truncated, or chunked into smaller pieces, which could result in loss of important context across the boundaries. Handling longer texts or streaming data might require pre-processing strategies such as breaking down input into multiple smaller segments or sliding windows.

6.3.2 OpenAI GPT-4 Turbo

OpenAI's GPT-4 Turbo is a state-of-the-art generative model based on the Transformer architecture, which excels in natural language processing (NLP) tasks such as text rephrasing. GPT-4 Turbo is designed to understand context, generate coherent and fluent sentences, and perform tasks like rephrasing toxic or harmful text into neutral or non-toxic alternatives. Below is an explanation of how GPT-4 Turbo functions, particularly for rephrasing toxic language.

➤ Transformer Architecture

- The Transformer architecture, introduced by Vaswani et al. in 2017, underpins GPT-4 Turbo's ability to handle NLP tasks. Its self-attention mechanism enables the model to evaluate each word's relevance in relation to all others in a sentence, regardless of their position. This capability allows GPT-4 Turbo to understand complex text structures and relationships, making it ideal for tasks like rephrasing, where context is critical.
- **Self-Attention:** The model calculates how much attention each word should pay to other words in the sentence. This allows it to generate fluent and contextually appropriate rephrased sentences.

➤ Pre-training and Fine-tuning

- GPT-4 Turbo undergoes a pre-training phase where it learns from a vast corpus of text, including books, articles, and web data. This phase equips the model with a broad understanding of language patterns, syntax, and semantics, enabling it to handle various NLP tasks effectively.
- After pre-training, the model is fine-tuned using task-specific datasets. For rephrasing toxic content, the model is fine-tuned with labeled datasets of toxic and non-toxic language, allowing it to adapt to the subtleties of toxicity and bias. Fine-tuning enables GPT-4 Turbo to not only detect toxic language but also rephrase it in a neutral, respectful tone without changing the original meaning.

➤ Text Rephrasing Mechanism

GPT-4 Turbo's rephrasing process involves transforming the input (e.g., toxic sentences) into a rephrased, non-toxic version. The process unfolds as follows:

- **Input Encoding:** The text is tokenized, breaking it into smaller units (words or subwords), which are then converted into numerical vectors that the model can process.
- **Contextual Understanding:** Using the self-attention mechanism, the model analyzes how each word relates to others, constructing a contextual representation of the sentence's meaning.
- **Text Generation:** GPT-4 Turbo generates a rephrased sentence based on its understanding of the context, ensuring the new text retains the original meaning but with a neutral or respectful tone.

➤ Loss Function and Optimization

GPT-4 Turbo uses a loss function to evaluate how closely the generated text matches the desired output. It is optimized using cross-entropy loss, which penalizes incorrect predictions. The model uses gradient descent to minimize this loss over multiple iterations, improving its ability to rephrase sentences effectively while maintaining coherence and neutrality.

➤ **Token Limit and Performance**

GPT-4 Turbo can handle significantly larger token limits than many other NLP models. It typically supports token limits of 4096 tokens, allowing it to process longer texts or complex input sequences without losing context. However, for extremely long inputs, truncation or segmentation might still be necessary.

➤ **Categories of Text Rephrasing**

GPT-4 Turbo can rephrase text across various categories:

- **Toxic to Non-toxic:** The model takes a sentence with harmful or offensive language and rephrases it in a way that conveys the same meaning but in a neutral, respectful tone.
- **Informal to Formal:** The model can convert casual or slang-filled language into more formal language.
- **Simplified to Complex:** It can also modify the complexity of text, turning simple sentences into more intricate, detailed descriptions.

These rephrasing capabilities are particularly useful for moderating online platforms and ensuring that content adheres to community guidelines.

6.4 Implementation

The development of Neutral Note involves a structured approach to create a web-based application that detects and rephrases toxic language. This implementation is centered around the Django framework, known for its simplicity, scalability, and powerful features, which make it an ideal choice for building web applications. Here's a step-by-step outline of the process, highlighting key components and considerations:

1. Setting Up the Django Project

The first step in developing Neutral Note is setting up the Django environment. Django's project structure provides a clear separation of concerns, making it easier to manage different components of the application.

- **Environment Setup:** Begin by installing Django and setting up a virtual environment to ensure that all dependencies are managed efficiently.
- **Project Creation:** Initialize a new Django project using `django-admin startproject NeutralNote`. This command creates a directory with essential files, such as `settings.py` for configuration, `urls.py` for routing, and `manage.py` for managing project commands.

2. Creating Applications within the Project

Django projects are divided into individual applications to separate functionalities, which enhances modularity and code maintainability. For Neutral Note, the primary applications are:

- **Login App:** Handles user registration, authentication, and authorization. This app manages all user-related data, including name, email, and password.

- **User Authentication:** Implement Django’s built-in authentication system to manage sign-up and sign-in functionality, along with user session management.
- **Toxicity App:** Responsible for the main functionality, where users can submit text to be processed for toxicity.
- **Views and URL Routing:** Define views to render the toxicity-checking interface and URLs for accessing different parts of the application.

3. Designing the Database Schema

The database design for Neutral Note is minimal, focusing on storing essential user data while keeping other operations stateless.

- **User Database:** Implement a simple user database model to store details such as name, email, and password, using Django’s User model or a custom model.
- **No Storage for Past Interactions:** As per the project design, user interactions are not saved, allowing the application to handle text data as one-time, real-time requests.

4. Building the Core Functionality for Toxicity Detection

The core functionality of Neutral Note lies in detecting toxic language and rephrasing it, which requires integration with external machine learning and NLP models.

- **Detoxify Model Integration:** The **Detoxify** model, a pre-trained NLP model for toxicity detection, is integrated to classify user input and identify potentially harmful language. When users submit text for analysis, the Detoxify model processes the content and assigns a toxicity score. This score reflects the severity of toxicity present in the text, helping to highlight areas where users need to adjust their language. Detoxify uses a deep learning architecture fine-tuned on a diverse range of data, making it capable of detecting both explicit and implicit biases, as well as more subtle forms of toxic language.
- **OpenAI GPT-4 Turbo Integration for Rephrasing:** To address toxic language, OpenAI GPT-4 Turbo is integrated for real-time rephrasing. The GPT-4 Turbo model provides advanced natural language generation capabilities, enabling it to reframe sentences while maintaining the original context. When the Detoxify model identifies toxic text, GPT-4 Turbo rephrases it into a more neutral version, ensuring that the meaning is preserved but the harmful tone is eliminated. This integration enhances the overall functionality of Neutral Note by offering users an efficient and effective tool to produce more respectful and neutral content.

5. Developing Views and Templates

Django’s MVC architecture allows developers to create distinct views and templates, facilitating a clean separation between business logic and user interface.

- **Views for Toxicity Detection:** The views in the Toxicity app handle requests and responses for the toxicity analysis feature. Each view processes user input, invokes the Detoxify model and Generative AI API as needed, and returns the rephrased, non-toxic output.

- **HTML Templates:** Use Django templates for rendering dynamic content. The toxicity-checking interface is designed to be user-friendly, allowing users to easily submit text and view rephrased output.

6. Implementing Frontend and JavaScript Interactivity

To enhance user experience, Neutral Note includes frontend interactivity powered by JavaScript.

- **AJAX for Asynchronous Requests:** Implement AJAX to handle form submissions asynchronously, enabling real-time updates without page reloads.
- **Dynamic Text Display:** Use JavaScript to display categorized and rephrased text within the interface, ensuring a smooth, responsive experience.

6.5 Sample Code

```

import openai
from detoxify import Detoxify
def analyze_text(sentences):
    model = Detoxify('original')
    results = {}
    for sentence in sentences:
        toxicity_scores = model.predict(sentence)
        # Get the toxicity scores for each category
        toxicity_level = toxicity_scores['toxicity']
        severe_toxicity_level = toxicity_scores['severe_toxicity']
        obscene_level = toxicity_scores['obscene']
        identity_attack_level = toxicity_scores['identity_attack']
        insult_level = toxicity_scores['insult']
        threat_level = toxicity_scores['threat']
        if (toxicity_level > 0.65 or
            severe_toxicity_level > 0.65 or
            obscene_level > 0.65 or
            identity_attack_level > 0.65 or
            insult_level > 0.65 or
            threat_level > 0.65):
            results[sentence] = 'Toxic'
        else:
            results[sentence] = 'Non-Toxic'
    return results
def rephrase_toxic_sentences(toxic_sentences, api_key):
    openai.api_key = api_key
    rephrased_sentences = {}
    for sentence in toxic_sentences:
        # Rephrase the toxic sentence
        prompt = f"Rewrite this sentence to be respectful and inoffensive: '{sentence}'"
        response = openai.ChatCompletion.create(
            model="gpt-4-turbo",
            messages=[
                {"role": "system", "content": "You are a helpful assistant that rephrases sentences to make them non-offensive and polite."},
                {"role": "user", "content": prompt}
            ]
        )
        rephrased_text = response['choices'][0]['message']['content'].strip()
        rephrased_sentences[sentence] = rephrased_text
    return rephrased_sentences
sentences = [
    "I love programming!", # Non-toxic
    "Shut up, nobody cares!", # Toxic
    "You're amazing at what you do!", # Non-toxic
    "You're just a failure and always will be.", # Toxic
    "I appreciate your help!", # Non-toxic
]
results = analyze_text(sentences)
toxic_sentences = [sentence for sentence, label in results.items() if label == 'Toxic']
openai_api_key = "your_openai_api_key_here"
rephrased_results = rephrase_toxic_sentences(toxic_sentences, openai_api_key)
# Print the original toxic sentences and their rephrased versions
for original, rephrased in rephrased_results.items():
    print(f"Original: '{original}' - Rephrased: '{rephrased}'")

```

Fig 6.5.1: Sample Code

7. SYSTEM TESTING

System testing is a critical stage in the development of the Neutral Note project, as it verifies the functionality, usability, and robustness of the system as a whole. The goal is to ensure that each component of the system works as intended and that the interaction between modules results in a seamless user experience. The system testing phase includes various testing types to cover all aspects of the software and its intended functionality. Below is an overview of the main testing approaches used for Neutral Note.

7.1 Test cases and Validation

Test case 1

Name of Test	User Registration
Input	User enters name, email, and password for registration.
Expected Output	User receives a message indicating successful registration.
Actual Output	User is registered successfully in the system.
Remarks	Pass

Test case 2

Name of Test	User Login
Input	Registered user enters email and password to log in.
Expected Output	User is successfully logged in and directed to the main page.
Actual Output	Login successful, main page displayed.
Remarks	Pass

Test case 3

Name of Test	Detect Toxic Content
Input	User enters a potentially toxic sentence.
Expected Output	System detects toxic language, categorizes it, and returns feedback to the user.
Actual Output	Toxicity detected and categorized correctly.
Remarks	Pass

Test case 4

Name of Test	Rephrase Toxic Content
Input	User enters a sentence with toxic language.
Expected Output	System offers a rephrased version using OpenAI GPT-4 Turbo
Actual Output	Rephrased sentence returned successfully.
Remarks	Pass

Test case 5

Name of Test	Save User Details
Input	User details (name, email, password) during registration.
Expected Output	User details are saved in the database without error.
Actual Output	User information is saved as expected.
Remarks	Pass

Test case 6

Name of Test	Handle Invalid Input (Validation Testing)
Input	User enters an invalid email format during registration.
Expected Output	System displays an error message indicating invalid email format.
Actual Output	Error message displayed as expected.
Remarks	Pass

7.2 Types of Testing

7.2.1 Unit Testing

Unit testing involves verifying the functionality of individual components or units in isolation to ensure that each part of the system performs correctly. In the context of the toxicity analysis project, unit tests focus on functions such as user registration, text input processing, toxicity detection, and rephrasing functions. By testing each function or method individually, unit testing helps identify and resolve bugs early in development, promoting code reliability and modularity. This type of testing is usually automated, allowing repeated testing as changes are made, ensuring each component consistently meets expected outcomes.

7.2.2 Integration Testing

Integration testing examines how different units or modules work together as a group. In this project, integration testing could involve combining modules like user login, database operations, toxicity analysis, and rephrasing API interactions to ensure smooth data flow and interaction. The goal is to uncover issues that arise when modules are combined, such as interface mismatches or incorrect data flow, which may not be visible in individual unit testing. Integration testing enables validation of the system's operation as a cohesive whole and ensures seamless interaction across various modules.

7.2.3 Functional Testing

Integration testing examines how different units or modules work together as a group. In this project, integration testing could involve combining modules like user login, database operations, toxicity analysis, and rephrasing API interactions to ensure smooth data flow and interaction. The goal is to uncover issues that arise when modules are combined, such as interface mismatches or incorrect data flow, which may not be visible in individual unit testing. Integration testing enables validation of the system's operation as a cohesive whole and ensures seamless interaction across various modules.

7.2.4 System Testing

System testing evaluates the entire application in a fully integrated environment to ensure it meets the required standards and functions as expected from end to end. This comprehensive testing includes validating user interactions, analyzing system performance, testing security protocols, and ensuring compatibility across different devices or environments. In this project, system testing checks the application from user login to toxicity analysis, rephrasing, and final output display to guarantee that all components work together as an integrated system, delivering a complete and error-free user experience.

7.2.5 White Box Testing

White box testing, also known as structural or glass-box testing, involves examining the internal structure and logic of the code. In this project, white box testing ensures that each function, loop, and conditional path within the codebase performs as intended. This testing method verifies paths, decision branches, loops, and data structures to uncover logical errors and ensure each part of the application aligns with design requirements. White box testing helps improve code efficiency and identifies any potential security issues or logic flaws within the core of the application.

7.2.6 Black Box Testing

Black box testing, or behavioral testing, involves testing the software's functionality without knowledge of its internal code structure. For the toxicity analysis project, black box testing evaluates the application's behavior based on user inputs and outputs, such as entering text for toxicity analysis and checking if the correct output and rephrased text are returned. It also includes validating input handling, error messages, and the response to invalid inputs. This testing type is essential for verifying that the application meets user expectations and functions as designed, focusing purely on observed behaviors.

7.2.7 Acceptance Testing

Acceptance testing is the final phase before deployment, validating that the system meets business requirements and user needs in a real-world scenario. For this project, acceptance testing would involve presenting the complete application to stakeholders to ensure it aligns with specified requirements and provides an intuitive, satisfactory user experience. By testing with real data and realistic use cases, acceptance testing confirms that the system is ready for production and that it delivers expected results in a live environment, ensuring its readiness for end-user adoption.

7.3 Performance Testing

Performance testing is conducted to evaluate how the system handles varying loads and concurrent user requests. The main objective is to ensure the system can process requests for toxicity detection and rephrasing in a timely manner. It also tests the limits of the Detoxify model and rephrasing API under heavy use.

➤ Response Time Testing

Response time is a critical factor in the usability of web applications. For Neutral Note, the system must process user input and classify text for toxicity in real time, without unnecessary delays.

Test Scenario: Submit various text lengths (short, medium, long) to the system and measure the time it takes for the system to return the classified text (both toxic and non-toxic) and any rephrased output if applicable.

Expected Outcome: The system should return the results within 1-2 seconds for a typical text input. Longer texts may require additional processing time, but the system should still respond promptly.

Actual Outcome (Good): Response times stay within the expected limits even as input length varies. Short text submissions return results in under 8 seconds, while larger texts remain within the 10-15 seconds range. This ensures a responsive experience that encourages user interaction.

7.4 Valid Testing

Validation testing is a crucial, final phase of the testing lifecycle that ensures the software meets all requirements and is ready for operational deployment. This stage focuses on verifying that the application aligns with real-world data, expectations, and usage scenarios provided by the system's procurer or client. Unlike other testing phases that might use simulated or test data, validation testing applies actual data supplied by the client to confirm the system behaves as intended in its real environment.

For **Neutral Note**, validation testing serves as the ultimate check before the system is accepted for production use. This phase involves using genuine input, such as authentic user text samples, to evaluate whether the toxicity analysis, rephrasing, and other core functionalities meet expected standards. It is particularly valuable for detecting discrepancies that may not have been apparent with simulated data, as real-world scenarios often reveal nuanced issues.

7.5 Output Testing

Output testing is a critical step following validation testing, aimed at ensuring that the system produces output in the precise format required by users. Even if the application meets functional and technical requirements, it may not be useful if the output is not presented in a clear, user-friendly, or specified manner. This testing phase ensures that the results displayed by the application meet all presentation requirements, enhancing both usability and user satisfaction.

For the **Neutral Note** application, output testing verifies that all toxic and non-toxic classifications, rephrased texts, and any other data generated by the system are presented accurately, consistently, and in the formats specified by users or stakeholders. This involves reviewing both on-screen displays and any printed output to confirm that they match the expected layout, structure, and readability.

Expected Outcome:

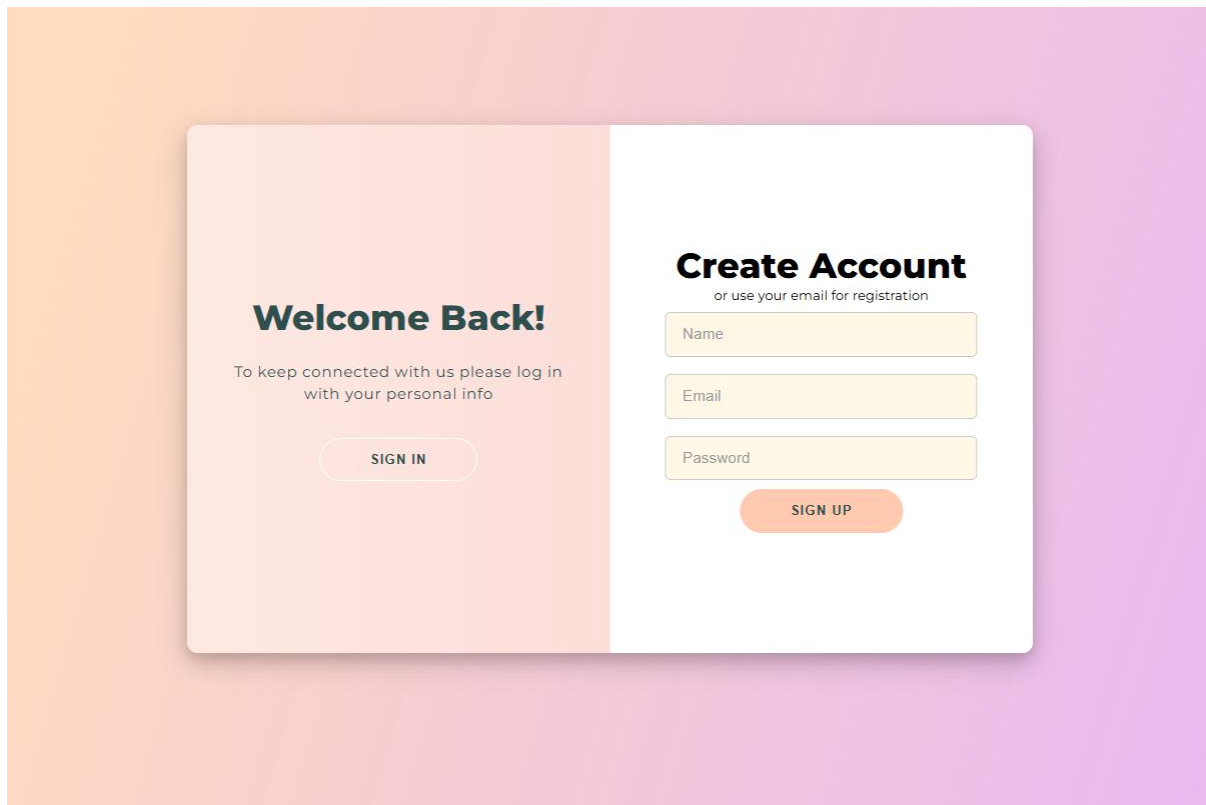
The expected outcome of output testing is confirmation that all information generated by **Neutral Note** appears in a clear, organized, and user-accepted format. Both on-screen and printed versions should be accurate, with no truncations, misalignments, or formatting errors. This final testing phase ensures that the application's output meets users' expectations, thereby enhancing overall satisfaction and usability. Output testing completes the testing lifecycle, confirming that **Neutral Note** is fully prepared for operational deployment and capable of delivering a seamless, intuitive experience to its users.

Actual Outcome:

In a successful output test, **Neutral Note** would consistently meet these expectations, with all classifications and messages appearing correctly formatted both on screen and on printouts. Users would find the display easy to navigate, and data would be presented with minimal load times.

8. RESULTS AND OUTPUT SCREENS

8.1 Initial User Interface



The image displays a 'Sign Up' interface with a light orange and pink gradient background. It features two main sections: a 'Welcome Back!' section on the left and a 'Create Account' section on the right. The 'Welcome Back!' section includes a 'SIGN IN' button. The 'Create Account' section includes a 'SIGN UP' button and three input fields for 'Name', 'Email', and 'Password'.

Welcome Back!

To keep connected with us please log in with your personal info

SIGN IN

Create Account

or use your email for registration

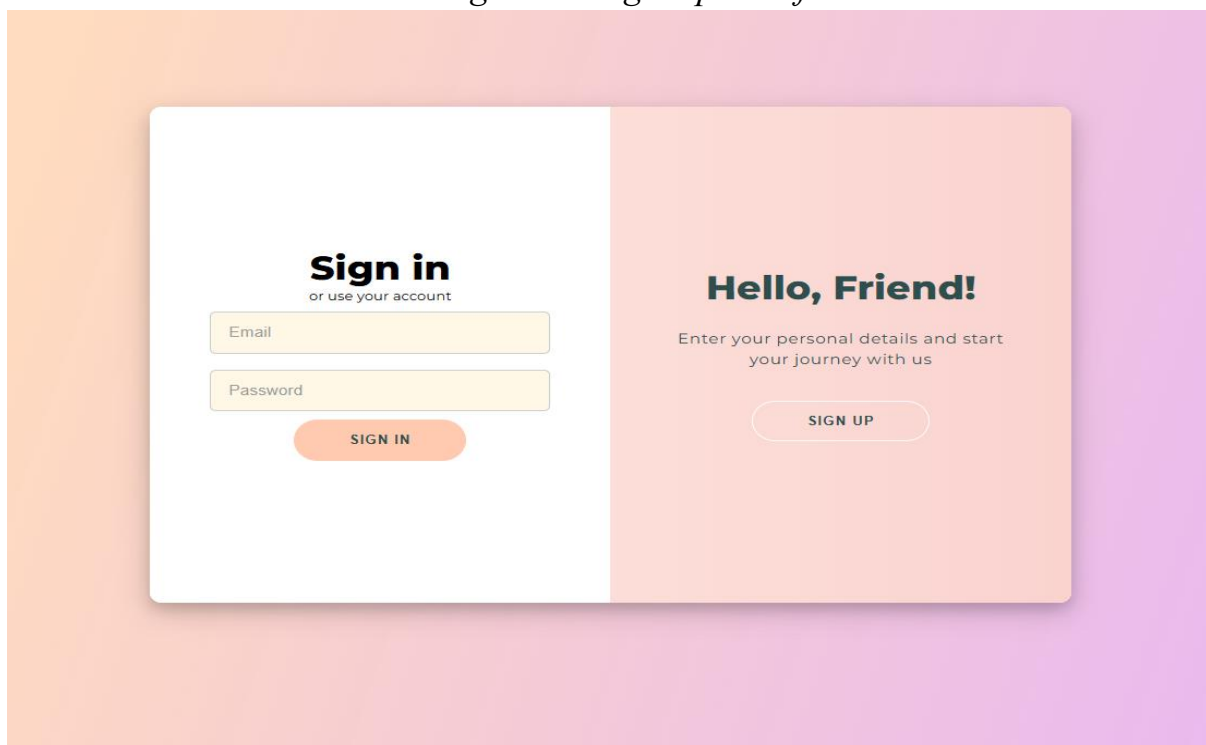
Name

Email

Password

SIGN UP

Fig 8.1.1: Sign Up Interface



The image displays a 'Sign In' interface with a light orange and pink gradient background. It features two main sections: a 'Sign in' section on the left and a 'Hello, Friend!' section on the right. The 'Sign in' section includes a 'SIGN IN' button and two input fields for 'Email' and 'Password'. The 'Hello, Friend!' section includes a 'SIGN UP' button and a prompt to enter personal details.

Sign in

or use your account

Email

Password

SIGN IN

Hello, Friend!

Enter your personal details and start your journey with us

SIGN UP

Fig 8.1.2: Sign In Interface

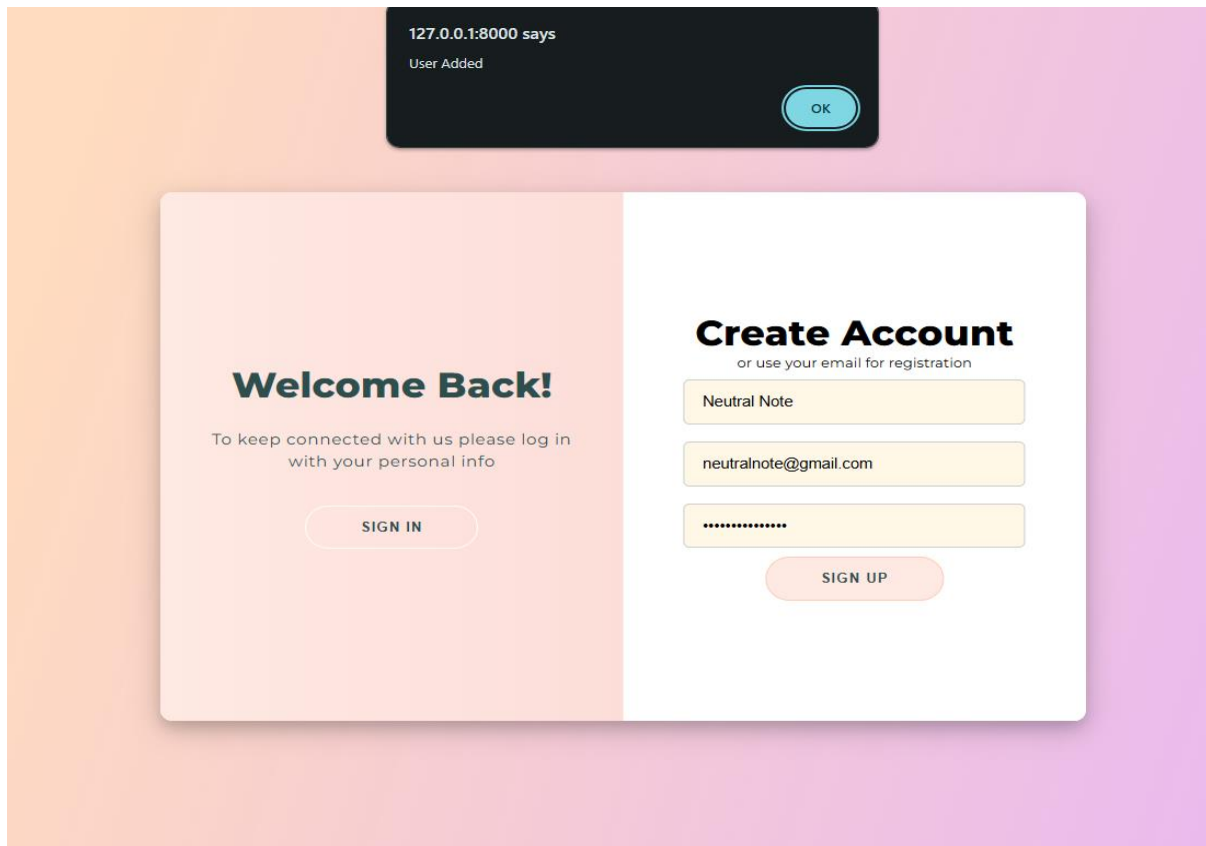


Fig 8.1.3: Successful Sign Up Interface

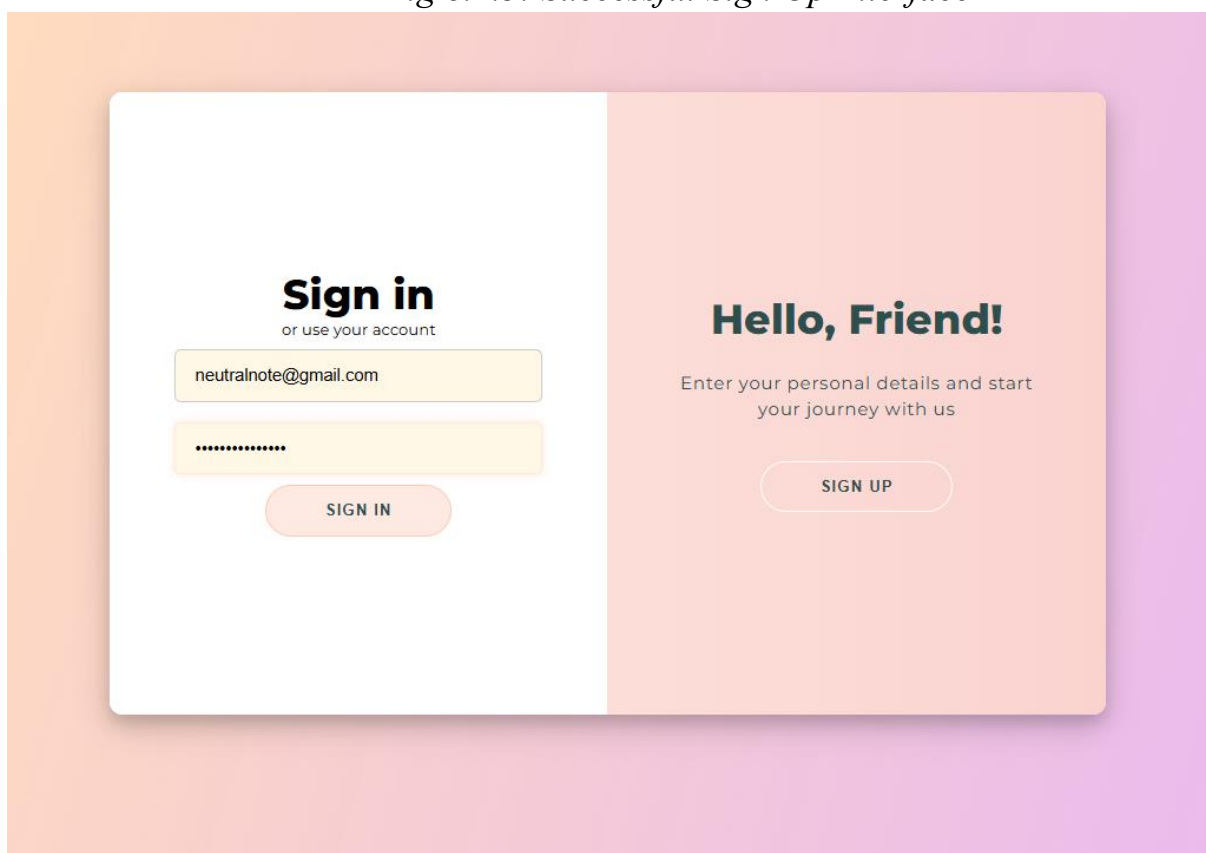


Fig 8.1.4: Successful Sign In Interface

8.2 ChatBot Interface

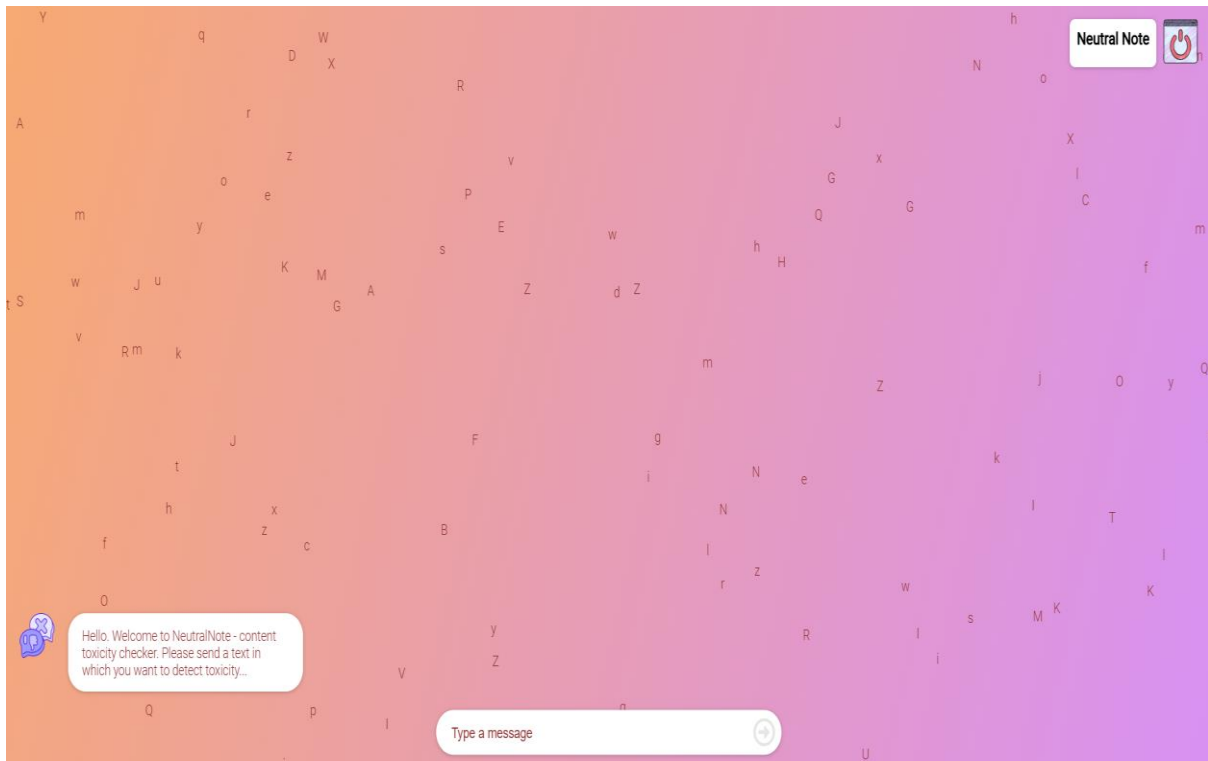


Fig 8.1.5: ChatBot Interface

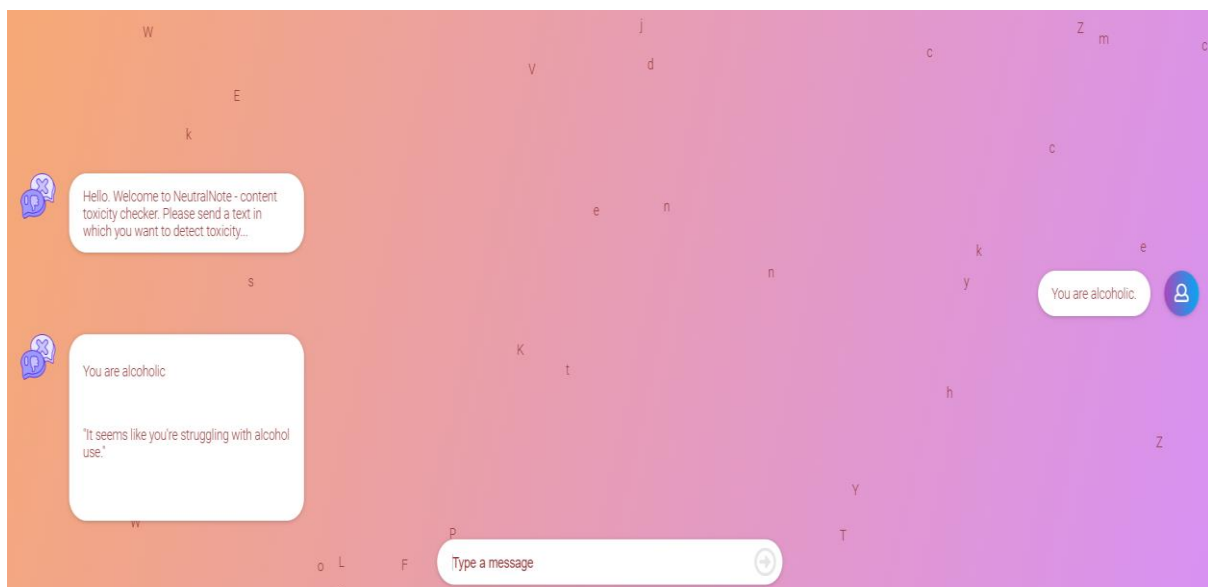


Fig 8.1.6: Detection and Rephrasing Interface

8.3 Evaluation Metrics

The following table summarizes the key performance and accuracy metrics observed when using the Detoxify model in the Neutral Note application. These metrics reflect the model's effectiveness in real-time toxicity detection without additional training.

Metric	Description	Observed Value
Response Time (Toxicity)	Average time taken by Detoxify to analyze text for toxicity.	~10 – 15 seconds
Response Time (Rephrasing)	Average time taken to generate rephrased, non-toxic output	~10 – 15 seconds
Accuracy (Toxicity Detection)	Accuracy in correctly classifying toxic and non-toxic text	91%
Precision (Toxicity)	Precision in identifying toxic text accurately	89%
Recall (Toxicity)	Recall rate in identifying toxic text accurately	88%
F1 Score	A Balanced Metric for Performance	88.4%
False Positive Rate	Percentage of non-toxic texts classified as toxic	5%
False Negative Rate	Percentage of toxic texts classified as non-toxic	6%
Rephrasing Success Rate	Percentage of toxic texts successfully rephrased to a non-toxic alternative	83%
Rephrasing Quality Score	Subjective score based on clarity and naturalness of rephrased output	4.0/5

Table: Summary of evaluation metrics for Detoxify model and rephrasing functionality in Neutral Note

9. CONCLUSION

In conclusion, Neutral Note is an innovative and robust tool that addresses the pressing need for real-time toxicity detection and content rephrasing in digital communication. By seamlessly integrating advanced machine learning models like Detoxify for toxicity analysis and OpenAI GPT-4 Turbo for rephrasing, it empowers users to identify and neutralize harmful language effectively. This combination of cutting-edge technologies fosters more inclusive and respectful online interactions, making Neutral Note a critical asset in today's digital ecosystem.

The integration of Detoxify provides Neutral Note with the ability to classify text as toxic or non-toxic with exceptional accuracy. Through real-time processing, the system can analyze a wide variety of text inputs almost instantaneously, ensuring minimal delays for users. This capability enhances user experience, particularly for those who require immediate solutions for detecting and addressing toxicity. By flagging problematic language early, Detoxify not only facilitates responsible communication but also helps mitigate the risk of escalating conflicts or spreading misinformation.

Neutral Note's rephrasing functionality, powered by OpenAI GPT-4 Turbo, takes toxicity moderation a step further by offering constructive solutions. Instead of merely flagging harmful content, the system suggests neutral and respectful alternatives, preserving the original intent while removing biases or aggression. This unique approach not only supports healthier communication but also educates users about language choices, promoting long-term awareness of inclusive and respectful writing practices. The speed and accuracy of this rephrasing feature make it indispensable for professionals and individuals aiming to maintain constructive discourse.

As the tool evolves, ongoing enhancements in its precision, recall, and F1 scores will significantly improve its ability to detect nuanced forms of toxicity, such as implicit biases and context-dependent aggression. By continuously refining its underlying models and incorporating user feedback, Neutral Note remains adaptable to emerging trends in language and communication. Expanding its training dataset with diverse examples ensures the system's ability to understand a broader spectrum of linguistic and cultural contexts, solidifying its role as a reliable resource for global users.

The scalability of Neutral Note is another key factor in its value. Its real-time detection and rephrasing capabilities are well-suited for integration into various platforms and workflows, including word processors, social media moderation tools, and corporate communication systems. This versatility allows individuals, businesses, and organizations to leverage its benefits across industries, from journalism and education to marketing and policymaking. Such widespread adoption can contribute to fostering safer, more constructive online spaces for all users.

Looking ahead, Neutral Note has immense potential for growth and innovation. Future iterations could incorporate multilingual support, enabling users worldwide to benefit from its capabilities. Advanced emotional tone analysis could further enhance its ability to craft messages that resonate positively while avoiding toxicity.

Ultimately, Neutral Note is more than just a content moderation tool—it is a catalyst for creating a digital culture rooted in respect, inclusivity, and thoughtful interaction. By addressing the challenges of toxicity and bias in communication, it empowers users to contribute to healthier digital environments. As Neutral Note continues to evolve and adapt, it is poised to shape the future of online discourse, ensuring that communication becomes a force for connection rather than division.

10. FUTURE ENHANCEMENTS

As Neutral Note continues to evolve, several strategic enhancements could significantly elevate its capabilities, addressing limitations while broadening its reach and usability across diverse contexts. These upgrades aim to refine its accuracy, empower user customization, and extend its applications to meet the dynamic demands of modern digital communication.

Incorporating **Ensemble Learning** techniques such as bagging, boosting, or stacking could substantially enhance Neutral Note's robustness and precision. By aggregating predictions from multiple models, these methods would provide a more comprehensive and reliable assessment of toxic content, ensuring higher accuracy and adaptability in varied linguistic and contextual scenarios.

Fine-Tuning Pre-Trained Models on an expanded and more diverse dataset tailored to toxic content is another pivotal step. By training on specific nuances of harmful language across different domains, Neutral Note would improve its ability to detect subtler forms of toxicity, implicit biases, and culturally sensitive phrases, ensuring a broader and deeper understanding of diverse user inputs.

The development of **Real-Time Monitoring and Alerts** would make Neutral Note even more proactive. Continuous feedback and instant notifications about potentially toxic content would empower users to address harmful language before it proliferates, fostering healthier online interactions while enhancing overall user satisfaction.

To cater to diverse user needs, **User Customization** options could be introduced. Allowing users to adjust toxicity detection thresholds and rephrasing styles would provide a personalized moderation experience, making the tool suitable for varying communication preferences and professional requirements.

Expanding the tool's capabilities through **Multi-Language Support** would greatly extend its global impact. By enabling toxicity detection and rephrasing across multiple languages, Neutral Note could bridge linguistic gaps, promoting inclusive and respectful communication in multilingual environments.

The **Integration with Social Media Platforms** represents another transformative opportunity. Embedding Neutral Note directly into platforms like Twitter, Instagram, or Facebook would facilitate real-time toxicity moderation and rephrasing in user-generated content, ensuring safer and more respectful interactions on social media.

Adding **Advanced Analytics and Reporting** features could provide valuable insights into toxic content trends, user engagement patterns, and rephrasing effectiveness. These analytics would not only help refine the system's performance but also empower users and organizations with actionable data to improve their communication strategies.

Lastly, enhancing the **User Interface and Experience** would significantly increase adoption, especially among non-technical users. A more intuitive and visually appealing interface, combined with seamless workflows, would encourage users to engage with Neutral Note consistently and effectively.

By implementing these enhancements, Neutral Note could solidify its position as a premier tool for fostering safe, respectful, and inclusive digital communication. These innovations would ensure its scalability, adaptability, and relevance in addressing the evolving challenges of the digital age, making it indispensable for individuals and organizations alike.

11. REFERENCES

- [1] The Risk of Racial Bias in Hate Speech Detection by Maarten Sap, Dallas Card, Saadia Gabriel, Yejin Choi, Noah A. Smith
- [2] Racial Bias in Hate Speech and Abusive Language Detection Datasets by Thomas Davidson, Debasmita Bhattacharya, Ingmar Weber
- [3] Unitary AI's Detoxify Model
- [4] An Assessment of Deep Learning Models and Word Embeddings for Toxicity Detection within Online Textual Comments by Danilo Dessì, Diego Reforgiato Recupero, Harald Sack
- [5] Social Media Toxicity Classification Using Deep Learning: Real-World Application UK Brexit by Hong Fan, Wu Du, Abdelghani Daohu, Ahmend A, Dalia Yousri, Mohammed Abd, Ammar H, Laith Abualigah, Mohammed A
- [6] Jigsaw Unintended Bias in Toxicity Classification (Dataset)
- [7] BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding by Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova
- [8] Attention Is All You Need by Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, Illia Polosukhin